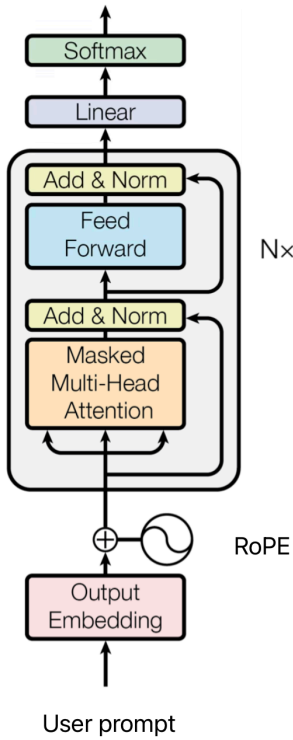


- **TiDAR Extension** - изследователски слой за hybrid decode и допълнителни loss/mask механизми.

Взаимодействието между модулите е организирано чрез shared конфигурации, стандартизирани entry points и общ artifact storage модел.

4.1. Базова архитектура на модела



Decoder-only архитектурата е организирана като последователност от attention + feed-forward блокове с residual връзки и нормализация между тях.

Входният текст се преобразува в embedding представяния, след което преминава през поредица от causal Transformer блокове. На изхода се получават logits върху речника, които чрез softmax формират вероятности за следващ token.

При training обработката е паралелна в рамките на контекстния прозорец, а при inference се използва prefill + decode режим с KV cache.

Figure 1: Decoder-only Transformer архитектура, използвана като базов модел

Базовият модел е decoder-only Transformer с causal masking. Всеки training sample се обработва като последователност от токени, а целевата функция е next-token prediction. Архитектурно са използвани съвременни компоненти като RMSNorm, RoPE и SwiGLU, което осигурява стабилност при обучение и добра ефективност при inference.

Multi-head attention операторът е:

$$h_h = \text{softmax} \left(\frac{(QW_h^Q)(KW_h^K)^T}{\sqrt{d_k}} + M \right) (VW_h^V)$$

$$\text{MHA}(Q, K, V) = [h_1 \parallel h_2 \parallel \dots \parallel h_H] W^O$$

където M е причинно-следствена маска, която гарантира, че позиция i вижда само токени с индекс $j \leq i$.

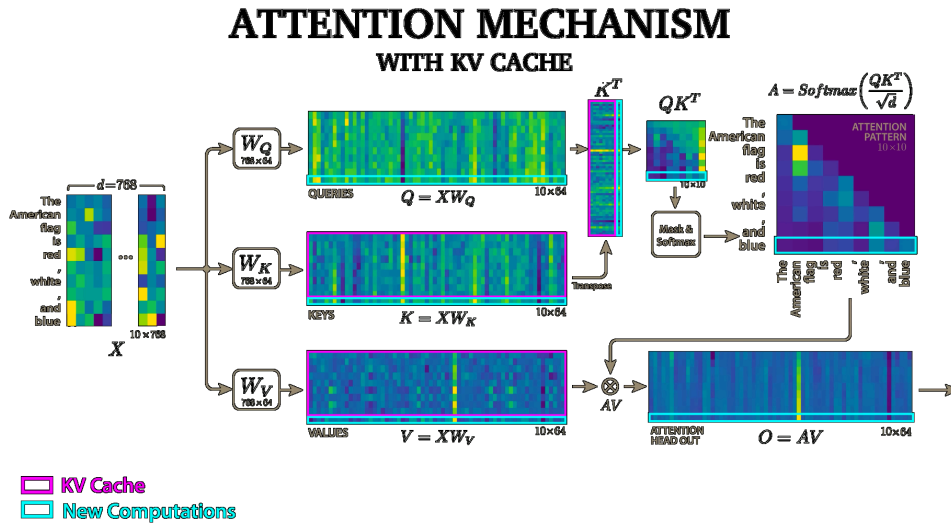


Figure 2: Attention механизъм и контекстно претегляне

4.2. Функционални модули и зависимости

Функционално решението е изградено от следните модули:

- **Data Module** - ingest, cleaning, dataset sharding, статистики, проверка на входните файлове.
- **Training Module** - optimizer loop, stages, gradient accumulation, checkpoint/recover.
- **Inference Module** - prefill/decode, sampling режими, KV cache, performance counters.
- **Evaluation Module** - benchmark сценарии и сравнение на ключови метрики.
- **Deployment Module** - Docker runtime, remote execution, data sync, artifact management.

Комуникацията между модулите е конфигурационно-ориентирана: модулите не разчитат на ръчно редактирани вътрешни параметри, а на explicit YAML settings, което намалява риска от скрити зависимости.

4.3. Data и training pipeline

Data pipeline изпълнява нормализация, филтриране, токенизация и shard-ване, а получените shard файлове се подават директно към training loop-а за предвидим I/O.

Training pipeline е stage-based: всеки stage има собствени параметри (контекст, learning schedule, loss weights), а състоянието се пази с checkpoint, което позволява надежден resume при прекъсване.

Базовите training термини се дефинират така:

$$L_A = -\frac{1}{N} \sum_{t=1}^N \ln p(y_t | y_{<t}), \quad L_D = -\frac{1}{N} \sum_{t=1}^N \ln p_{\theta}(y_t | c_t)$$

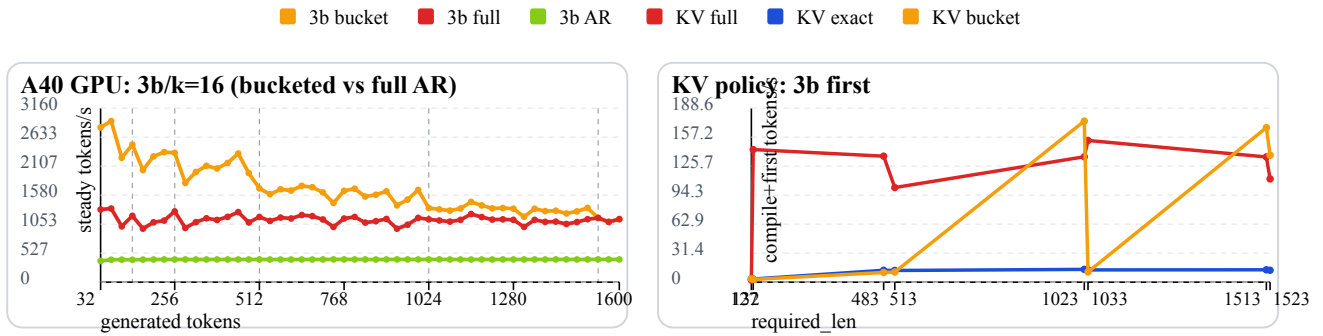
където L_A е autoregressive cross-entropy термин, L_D е diffusion/draft термин, N е броят обучителни позиции в batch-a, y_t е таргет токенът на позиция t , а c_t е контекстът, достъпен за diffusion клона при съответната маска.

Общата loss комбинация в Anchor-TiDAR режима е:

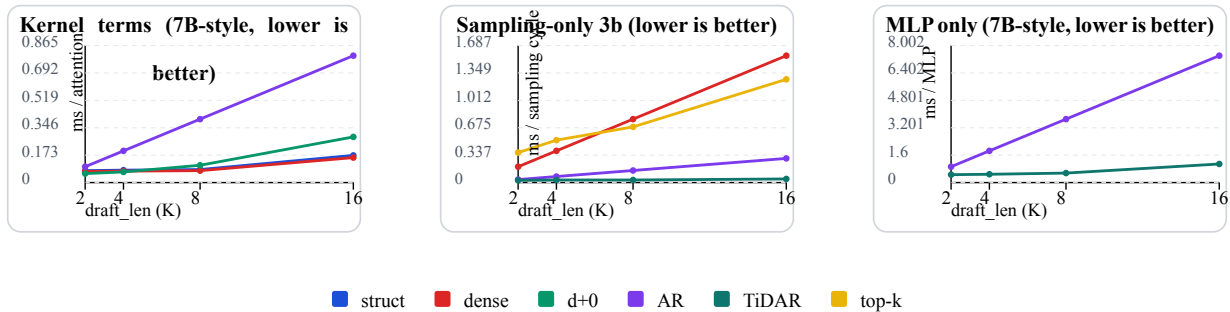
$$L = \alpha L_A + \beta L_D + \rho D_f + \chi D_r + \delta L_H + \delta_m L_P + \eta L_S + \gamma L_K$$

В практическата постановка се използват няколко вида loss термини: (1) базови езикови термини за AR и Diff клоновете, (2) дистрибуционни термини за подравняване между двата клона, (3) маскирани/префиксни термини за стабилност на acceptance веригата и (4) допълнителни регуляризационни термини за контрол на training динамиката.

$$D_f = \sum_v p_A(v) \ln \frac{p_A(v)}{p_D(v)}, \quad D_r = \sum_v p_D(v) \ln \frac{p_D(v)}{p_A(v)}, \quad L_H = - \sum_t m_t \ln p_D(y_t | c_t)$$



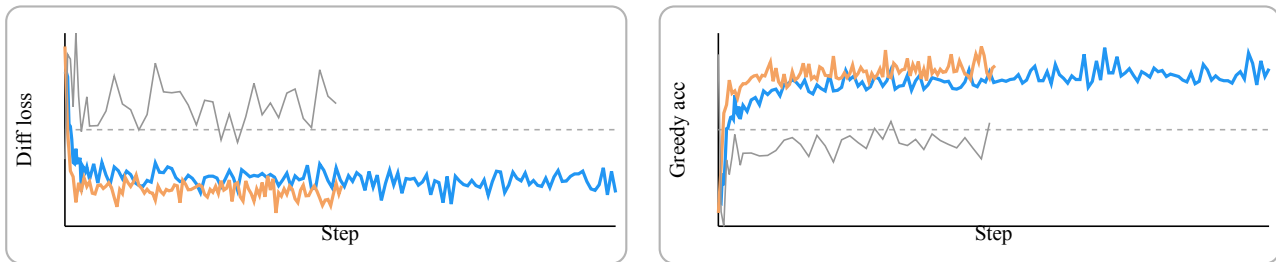
Двете графики показват как политиката за оразмеряване на KV cache влияе директно върху decode производителността: bucketed подходът запазва по-висока steady скорост спрямо full-context baseline, а в first-run режима се вижда цената на compile и ефектът от cache sizing върху първата заявка.



Тези 3 графики показват Free Token Slots феномена при attention, sampling и MLP.

SmolLM 135M vs 360M

135M 360M ratio 135/360



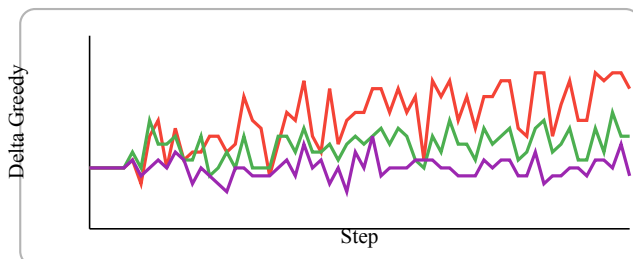
По-големият 360M модел поддържа по-стабилен quality профил при сравним decode режим, което е очакван индикатор за по-висок моделен капацитет.

В practical inference план това означава, че 360M дава по-предсказуемо поведение при сходни decode настройки, докато 135M остава полезен за по-евтини и бързи итерации. Тази двойка графики е полезна за избор на model size спрямо budget/quality изисквания.

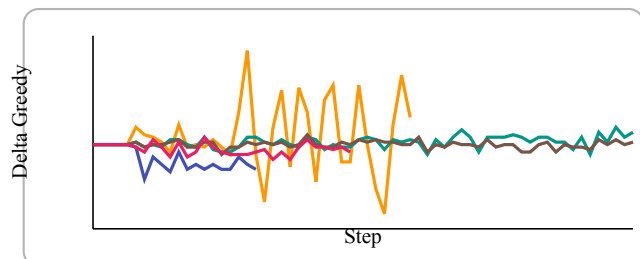
Stable baseline KL-only no-diff KL-keep soft-align Distill AR->Diff
G-eta hard-agree B-beta diff-boost Top-K set-match B-gamma hard+set D-mask prefix-only

Следващите графики показват delta спрямо stable baseline за core и за разширения набор от loss конфигурации върху TinyStories (90k-121k).

Core loss варианти



Разширен loss набор



Анализ: TinyStories е нискоентропиен корпус с ограничена лексикална вариативност, затова разликите между loss вариантите често са с малка амплитуда. За по-ясно разделяне на ефектите е необходимо скалиране към по-големи модели и по-трудни корпуси.

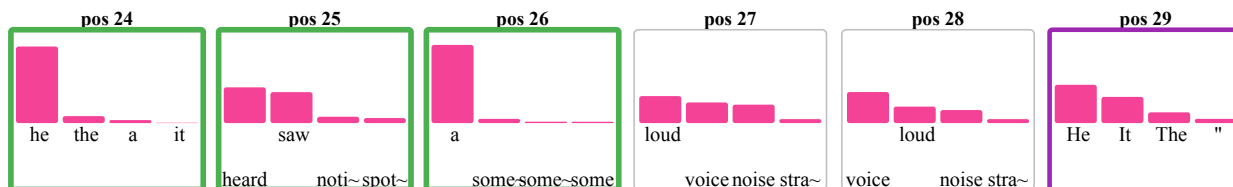
Core вариантите са добър избор за стабилност и контрол, докато разширеният набор е по-подходящ за търсене на агресивни speedup хипотези.

Логитни хистограми (TinyStories примери)

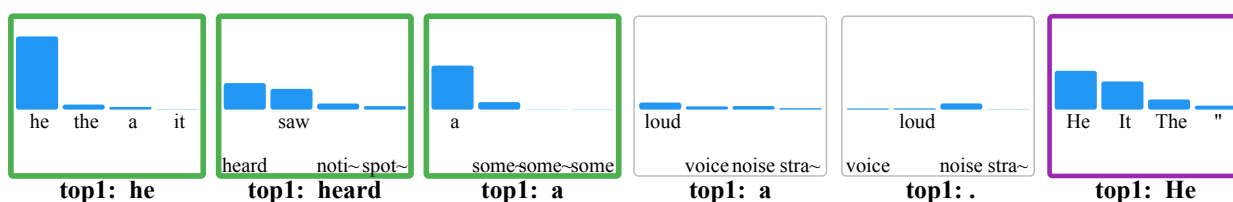
За всяка позиция са показани два панела: горният е AR verify разпределението, а долният е Diff draft разпределението за същите токени в същия ред.

Prompt: “In the forest, a tiny fox”

AR verify logits (top-4, sorted by AR)



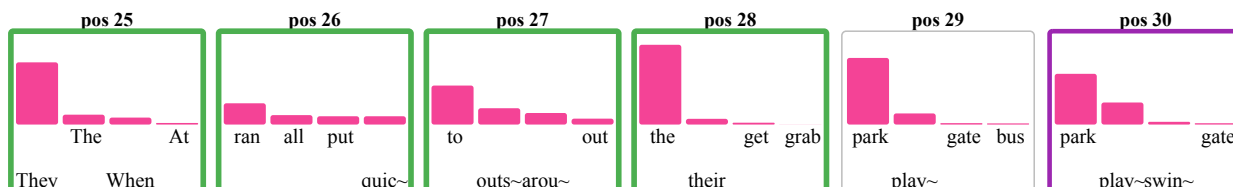
Diff draft logits (same tokens, AR order)



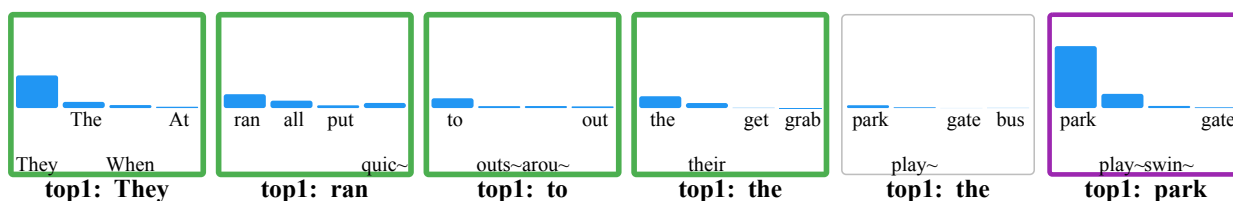
Sentence context Suddenly, he heard a loud noise.

Prompt: “One day, the teacher said”

AR verify logits (top-4, sorted by AR)



Diff draft logits (same tokens, AR order)



Sentence context They ran to the park and saw a big slide.

След първия reject често се наблюдават локални top-1 съвпадения между AR и Diff, но acceptance веригата остава прекъсната за текущата итерация, поради което позициите не се маркират като приети.

4.4. Anchor-TiDAR: hybrid decode логика

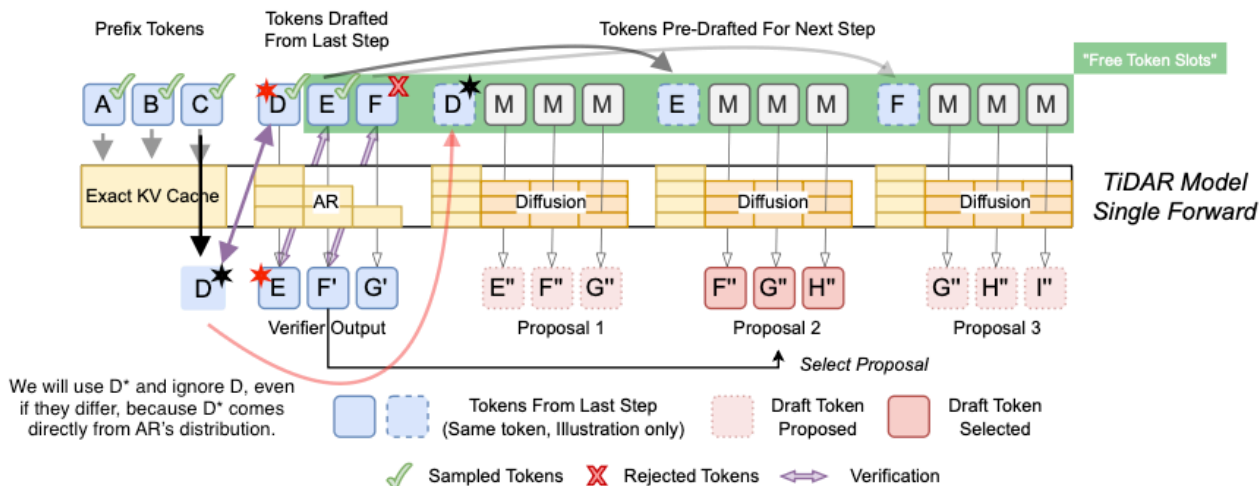


Figure 3: Anchor-TiDAR forward pass: verify + predraft в един моделен проход

Anchor-TiDAR въвежда sequence-level hybrid decode: системата генерира draft предложения паралелно и ги верифицира autoregressively в рамките на един forward pass със специализирани маски. Това позволява по-добро използване на GPU паралелизма спрямо класическия AR decode път.

Логически decode цикълът е:

1. prefill на контекста и инициализация на KV cache;
2. генериране на draft кандидати;
3. verify стъпка и приемане/отхвърляне на префикс;
4. commit/rollback на pointer-и в KV cache;
5. преход към следваща итерация.

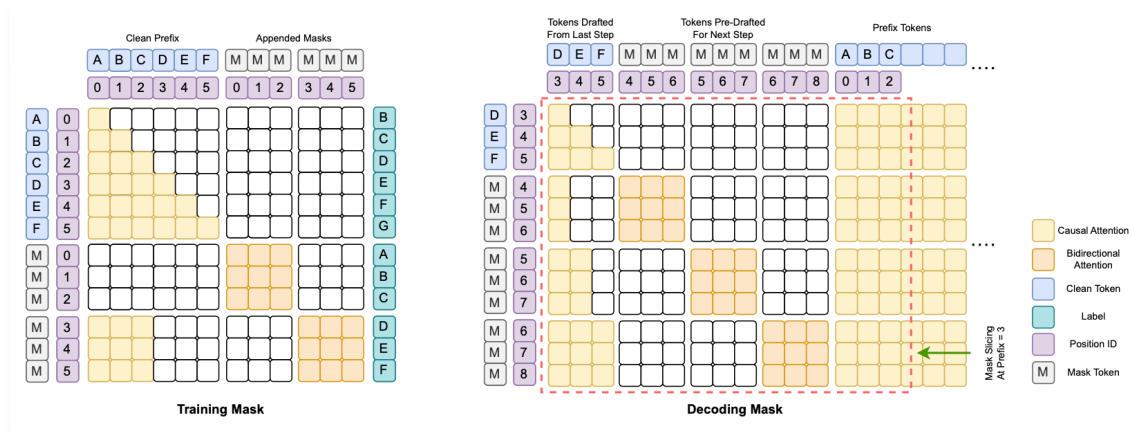


Figure 4: TiDAR маски: decode и training с обща легенда

Практическата полза е, че verify/predraft изчисленията се амортизират в един компактен execution path, което намалява относителния overhead при decode.

4.5. Free Token Slots и GPU ефективност

Концепцията Free Token Slots описва случаи, при които в дадена decode итерация има неизползван паралелен ресурс, който може да бъде запълнен с полезна допълнителна работа. Този анализ е ключов за latency-critical deployment сценарии, защото показва къде архитектурните промени носят реална speedup стойност.

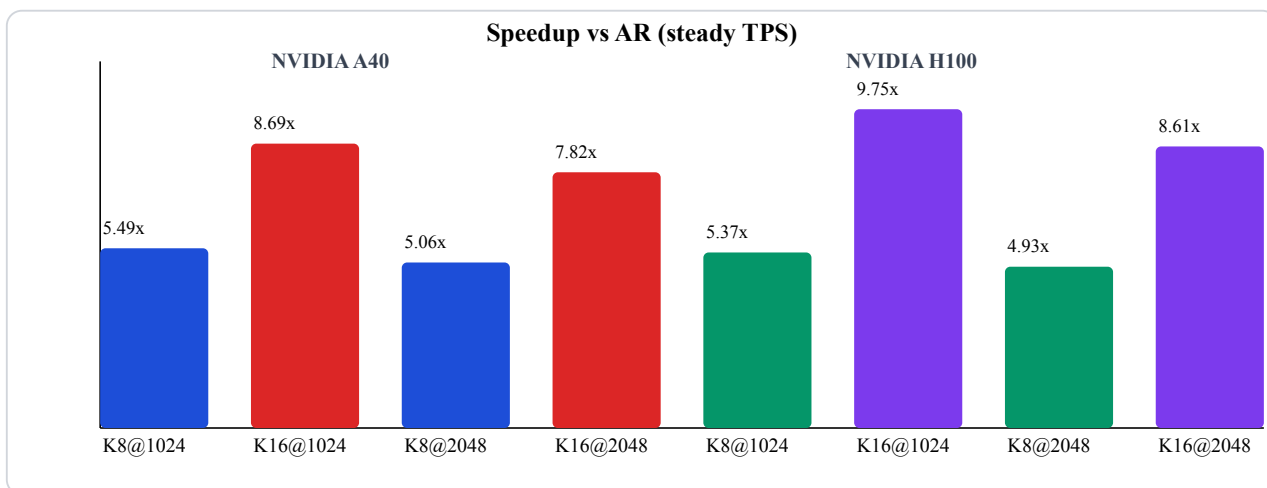
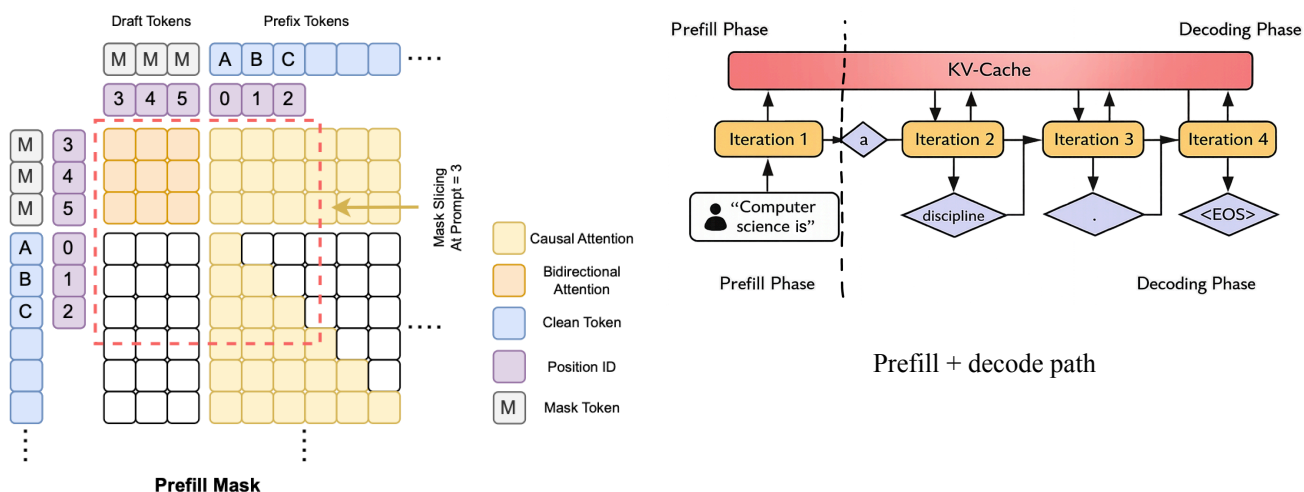


Figure 5: Head-to-head speedup диаграма (steady TPS), рендерирана директно от Typst

Валидираните резултати в този проект показват, че hybrid decode подходът е устойчив при мащаби до 7B параметри и може да осигури значимо ускорение спрямо AR baseline при запазване на конкурентно качество.

4.6. End-to-end decode път (prefill + decode)



TiDAR prefill mask

Figure 6: End-to-end decode път в GIANT/TiDAR: prefill маска (ляво) и decode pipeline (дясно)
Практически inference изпълнението е разделено на две фази. В **prefill** фазата целият prompt се обработва наведнъж, като се инициализира KV cache състоянието за последващото генериране.

В **decode** фазата системата добавя нови токени итеративно, като използва кешираната история вместо повторно пресмятане на целия контекст.

При GIANT този процес следва класически AR модел. При TiDAR/Anchor-TiDAR към същия execution skeleton се добавят verify/predraft стъпки и структурирани маски, без да се нарушава базовата консистентност на KV cache. Това позволява сравнение между режимите в една и съща инфраструктура и при едни и същи входни условия.

Поради тази унифицирана структура измерванията за скорост, закъснение и стабилност са директно съпоставими между AR и hybrid decode вариантите, което е критично за обективна оценка на реалния practically usable speedup.

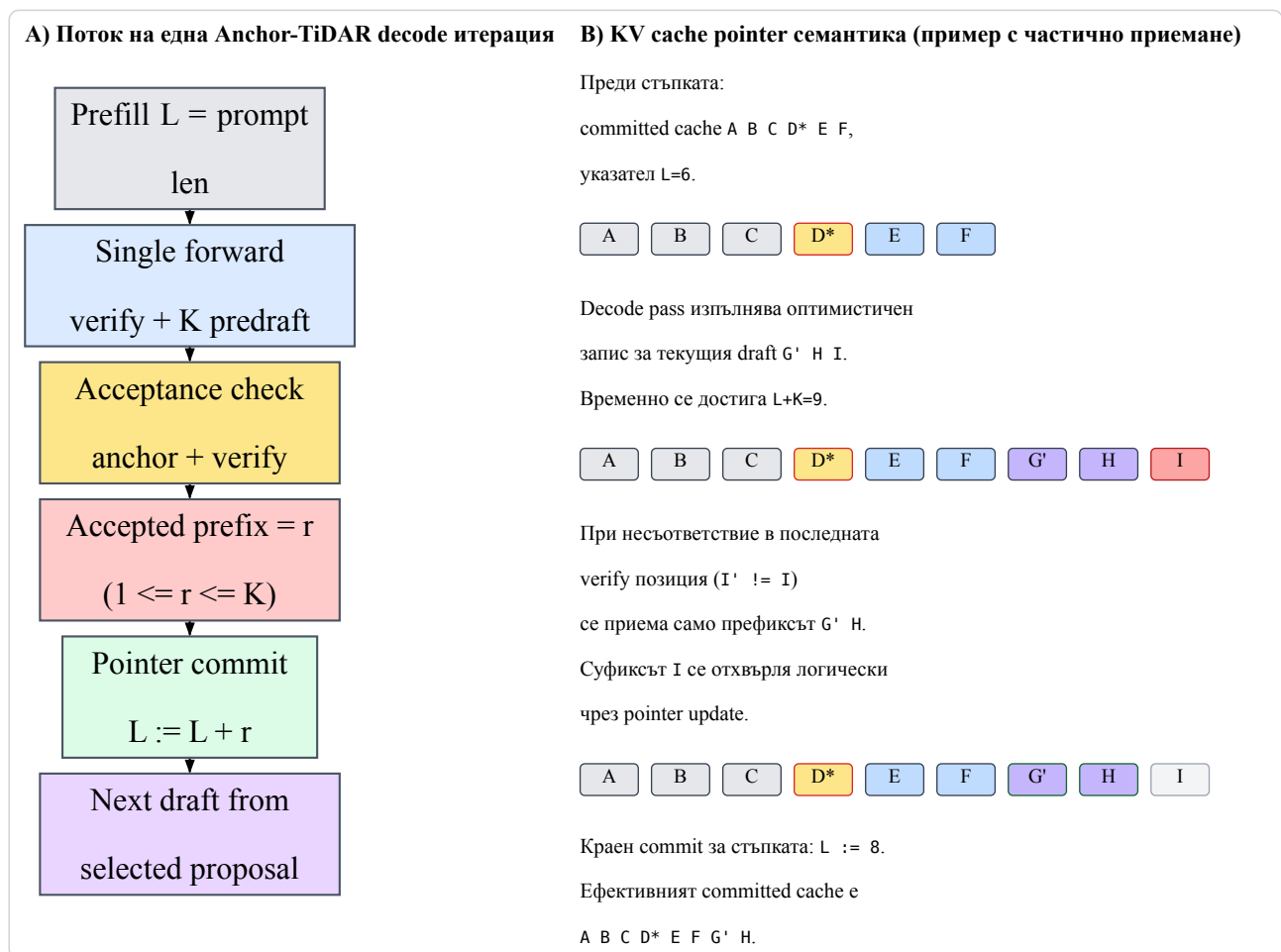


Figure 7: KV cache commit/rollback при Anchor-TiDAR: поток и pointer update